

scripts/08_export_report_pdf.py

231 lines

```
1 """
2 08_export_report_pdf.py
3 =====
4 Renders `report/ENGINEERING_REPORT.md` to a polished PDF at
5 `submission/ENGINEERING_REPORT.pdf`, for the hackathon's "pdf or zip only"
6 upload requirement.
7
8 This is a small purpose-built Markdown -> ReportLab converter (headings,
9 tables, bullet/numbered lists, bold/inline-code spans, images, horizontal
10 rules, fenced code blocks) rather than a generic HTML renderer - it only
11 needs to handle the constructs actually used in our one report file, and
12 keeping it dependency-light (reportlab + markdown's tiny inline regexes only)
13 avoids pulling in a headless-browser PDF toolchain that may not be available
14 in the judging environment.
15
16 Requires: pip install reportlab markdown (NOT needed for the core pipeline;
17 only for this optional PDF export.)
18 """
19
20 from __future__ import annotations
21
22 import re
23 import sys
24 from pathlib import Path
25
26 from reportlab.lib import colors
27 from reportlab.lib.pagesizes import LETTER
28 from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
29 from reportlab.lib.units import inch
30 from reportlab.platypus import (
31 SimpleDocTemplate, Paragraph, Spacer, Image, Table, TableStyle,
32 HRFlowable, ListFlowable, ListItem, PageBreak, KeepTogether,
33 )
34
35 ROOT = Path(__file__).resolve().parents[1]
36 REPORT_MD = ROOT / "report" / "ENGINEERING_REPORT.md"
37 OUT_DIR = ROOT / "submission"
38 OUT_PDF = OUT_DIR / "ENGINEERING_REPORT.pdf"
39
40 INK = colors.HexColor("#111827")
41 DIM = colors.HexColor("#6b7280")
42 BLUE = colors.HexColor("#2563eb")
43 LINE = colors.HexColor("#e5e7eb")
44 HEADFILL = colors.HexColor("#f3f4f6")
45
46 styles = getSampleStyleSheet()
47 STYLES = {
48 "title": ParagraphStyle("title", parent=styles["Title"], fontSize=20, textColor=INK, spaceAfter=4),
49 "subtitle": ParagraphStyle("subtitle", parent=styles["Normal"], fontSize=11, textColor=DIM, spaceAfter=18),
50 "h2": ParagraphStyle("h2", parent=styles["Heading1"], fontSize=15, textColor=INK,
51 spaceBefore=18, spaceAfter=8, borderColor=LINE),
52 "h3": ParagraphStyle("h3", parent=styles["Heading2"], fontSize=12.5, textColor=BLUE,
53 spaceBefore=12, spaceAfter=6),
54 "body": ParagraphStyle("body", parent=styles["Normal"], fontSize=9.7, leading=14,
55 textColor=INK, spaceAfter=6),
56 "bullet": ParagraphStyle("bullet", parent=styles["Normal"], fontSize=9.7, leading=14, textColor=INK),
57 "cell": ParagraphStyle("cell", parent=styles["Normal"], fontSize=8.3, leading=11, textColor=INK),
58 "cellhead": ParagraphStyle("cellhead", parent=styles["Normal"], fontSize=8.3, leading=11,
59 textColor=colors.white, fontName="Helvetica-Bold"),
60 "code": ParagraphStyle("code", parent=styles["Normal"], fontName="Courier", fontSize=8.2,
61 leading=11, textColor=INK, backColor=HEADFILL,
62 borderPadding=8, spaceAfter=8),
63 "quote": ParagraphStyle("quote", parent=styles["Normal"], fontSize=9.4, leading=14,
64 textColor=DIM, leftIndent=18, rightIndent=10,
65 spaceBefore=4, spaceAfter=8, borderPadding=6,
66 fontName="Helvetica-Oblique"),
67 "caption": ParagraphStyle("caption", parent=styles["Normal"], fontSize=8.3, textColor=DIM,
```

```

68 alignment=1, spaceAfter=12),
69 }
70
71
72 def inline(text: str) -> str:
73 """Convert a subset of inline Markdown to ReportLab's mini-XML markup."""
74 text = text.replace("&", "&").replace("<", "<").replace(">", ">")
75 # restore markup we want to allow through after escaping
76 text = re.sub(r"\*(.+?)\*", r"<b>\1</b>", text)
77 text = re.sub(r"(?!\\w)\*([^\*]+)\*(?!\\w)", r"<i>\1</i>", text)
78 text = re.sub(r"`([^\`]+)`", r"<font face='Courier' size='8.3' color='#b91c1c'>\1</font>", text)
79 text = re.sub(r"\[([^\]]+)\]\[([^\]]+)\]", r"\1", text) # links -> plain text (local anchors)
80 return text
81
82
83 def parse_table(lines: list[str]) -> Table:
84 rows = [l.strip().strip("|").split("|") for l in lines if not re.match(r"^\s*\|?\s*-\s*\|", l)]
85 rows = [[c.strip() for c in row] for row in rows]
86 data = []
87 for ri, row in enumerate(rows):
88 style = STYLES["cellhead"] if ri == 0 else STYLES["cell"]
89 data.append([Paragraph(inline(c), style) for c in row])
90 ncols = len(data[0])
91 avail_width = 6.6 * inch
92 col_w = avail_width / ncols
93 t = Table(data, colWidths=[col_w] * ncols, repeatRows=1)
94 t.setStyle(TableStyle([
95 ("BACKGROUND", (0, 0), (-1, 0), BLUE),
96 ("BACKGROUND", (0, 1), (-1, -1), colors.white),
97 ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#f9fafb")])),
98 ("GRID", (0, 0), (-1, -1), 0.5, LINE),
99 ("VALIGN", (0, 0), (-1, -1), "TOP"),
100 ("TOPPADDING", (0, 0), (-1, -1), 4),
101 ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
102 ("LEFTPADDING", (0, 0), (-1, -1), 5),
103 ("RIGHTPADDING", (0, 0), (-1, -1), 5),
104 ]))
105 return t
106
107
108 def build_story(md_text: str) -> list:
109 story = []
110 lines = md_text.split("\n")
111 i = 0
112 first_h1 = True
113 while i < len(lines):
114 line = lines[i]
115
116 if line.startswith("# "):
117 if first_h1:
118 story.append(Paragraph(inline(line[2:]), STYLES["title"]))
119 first_h1 = False
120 else:
121 story.append(Paragraph(inline(line[2:]), STYLES["h2"]))
122 i += 1
123 elif line.startswith("## "):
124 story.append(Paragraph(inline(line[3:]), STYLES["h2"]))
125 i += 1
126 elif line.startswith("### "):
127 story.append(Paragraph(inline(line[4:]), STYLES["h3"]))
128 i += 1
129 elif line.startswith("***Challenge:***") or line.startswith("***Team submission***"):
130 story.append(Paragraph(inline(line), STYLES["subtitle"]))
131 i += 1
132 elif line.strip() == "---":
133 story.append(Spacer(1, 4))
134 story.append(HRFlowable(width="100%", thickness=0.6, color=LINE, spaceAfter=10))
135 i += 1
136 elif line.strip().startswith("`"):
137 i += 1

```

```

138 code_lines = []
139 while i < len(lines) and not lines[i].strip().startswith("`"):
140     code_lines.append(lines[i])
141     i += 1
142     i += 1 # skip closing fence
143     code_txt = "<br/>".join(l.replace(" ", "&nbsp;") or "&nbsp;" for l in code_lines)
144     story.append(Paragraph(code_txt, STYLES["code"]))
145     elif line.strip().startswith("!"):
146         m = re.match(r"!\\([\\^\\]*)\\((\\^)+\\)", line.strip())
147         if m:
148             alt, relpath = m.groups()
149             img_path = (REPORT_MD.parent / relpath).resolve()
150             if img_path.exists():
151                 img = Image(str(img_path), width=6.3 * inch, height=6.3 * inch * 0.62)
152                 img.hAlign = "CENTER"
153                 story.append(img)
154                 story.append(Paragraph(alt, STYLES["caption"]))
155                 i += 1
156         elif line.strip().startswith(">"):
157             quote_lines = []
158             while i < len(lines) and lines[i].strip().startswith(">"):
159                 quote_lines.append(re.sub(r"^\\s*>\\s?", "", lines[i]))
160                 i += 1
161             text = " ".join(l.strip() for l in quote_lines if l.strip())
162             if text:
163                 story.append(Paragraph(inline(text), STYLES["quote"]))
164             elif line.strip().startswith("|"):
165                 table_lines = []

166         while i < len(lines) and lines[i].strip().startswith("|"):
167             table_lines.append(lines[i])
168             i += 1
169         story.append(Spacer(1, 4))
170         story.append(parse_table(table_lines))
171         story.append(Spacer(1, 10))
172         elif re.match(r"^\\s*-\\s+", line):
173             items = []
174             while i < len(lines) and re.match(r"^\\s*-\\s+", lines[i]):
175                 item_lines = [re.sub(r"^\\s*-\\s+", "", lines[i])]
176                 i += 1
177             while i < len(lines) and lines[i].strip() != "" and not re.match(
178                 r"^\\s*(-\\s+|\\d+\\.\\s+|#|\\|`|!\\[|---)", lines[i]
179             ):
180                 item_lines.append(lines[i].strip())
181                 i += 1
182             items.append(ListItem(Paragraph(inline(" ".join(item_lines))), STYLES["bullet"]),
183                               bulletColor=BLUE))
184             story.append(ListFlowable(items, bulletType="bullet", start="circle", leftIndent=16))
185             story.append(Spacer(1, 6))
186         elif re.match(r"^\\s*d+\\.\\s+", line):
187             items = []
188             while i < len(lines) and re.match(r"^\\s*d+\\.\\s+", lines[i]):
189                 item_lines = [re.sub(r"^\\s*d+\\.\\s+", "", lines[i])]
190                 i += 1
191             while i < len(lines) and lines[i].strip() != "" and not re.match(
192                 r"^\\s*(-\\s+|\\d+\\.\\s+|#|\\|`|!\\[|---)", lines[i]
193             ):
194                 item_lines.append(lines[i].strip())
195                 i += 1
196             items.append(ListItem(Paragraph(inline(" ".join(item_lines))), STYLES["bullet"])))
197             story.append(ListFlowable(items, bulletType="1", leftIndent=16))
198             story.append(Spacer(1, 6))
199         elif line.strip() == "":
200             i += 1
201         else:
202             para_lines = [line]
203             i += 1
204             while i < len(lines) and lines[i].strip() != "" and not re.match(
205                 r"^(#|>|\\d+\\.\\s+|`|!\\[|---)", lines[i].strip()
206             ):
207                 para_lines.append(lines[i])

```

```
208 i += 1
209 story.append(Paragraph(inline(" ".join(para_lines)), STYLES["body"]))
210
211 return story
212
213
214 def main():
215     OUT_DIR.mkdir(exist_ok=True)
216     md_text = REPORT_MD.read_text()
217     story = build_story(md_text)
218
219     doc = SimpleDocTemplate(
220         str(OUT_PDF), pagesize=LETTER,
221         topMargin=0.65 * inch, bottomMargin=0.65 * inch,
222         leftMargin=0.7 * inch, rightMargin=0.7 * inch,
223         title="Autonomous Production Choke Controller – Engineering Report",
224     )
225     doc.build(story)
226     print(f"wrote {OUT_PDF} ({OUT_PDF.stat().st_size/1024:.0f} KB)")
227
228
229 if __name__ == "__main__":
230     main()
231
```